

Codex 5.5 Pro Audit Prompt: Engine_1 Multi-Strategy Integration

You are an elite quantitative systems architect and lead compiler/security auditor. Your task is to perform a rigorous line-by-line validation, security audit, and behavioral analysis of our production-grade dual-strategy execution engine ('Engine_1.py').

```

The engine runs concurrently on a **MetaTrader 5 (MT5) Blueberry
Funded Account** under strict risk controls:
--**Max Daily Drawdown (DD) Limit** 5% (Hard breach).
--**Max Total DD Limit** 10% (Hard breach).
--**Asset Universe** High-leverage crypto perpetuals (BTCUSD,
ETHUSD, SOLUSD, BNBUSD, SUIUSD, etc.).
--**Coinbase Scraped WebSocket Feed + Binance Footprint WebSocket
Feed** delivering ticks into an asynchronous in-memory data store
('SnapshotStore').

```

1000

1. STRATEGY CONTEXT

The engine implements exactly two strategies:

1. **AlphaSqueezer_V17**: A trend-following classifier ensemble (LGBM, XGBoost, CatBoost) running on 15m closed candle boundaries.
2. **ML_Liquidation_Runner**: An extreme-event reversal/breakout ensemble evaluating liquidations. It uses a 4H Trend Confluence Filter and an Equity Curve Circuit Breaker.

2. INTEGRATED ENGINE CODE TO AUDIT

Component A: The Live Liquidation Predictor

```
python
class LiveLiquidationPredictor:
    def __init__(self, syr ↓, List[str]):
        self.symbols = symbols
        self.models = {}
        self.candles_history = {}
        self.current_candle = {}
        self._lock = threading.RLock()
        # Equity curve tracking — rolling window of recent capital levels (OOS
```

```
self.recent_capitals = [5000.0]
self._compute_rolling_features = None # lazy-loaded once
self.latest_atr = {}
```

```
self.load_models()
```

```
def load_models(self):
    import pickle
    import sys
    base_dir = os.path.dirname(os.path.abspath(__file__))
    models_dir = os.path.join(base_dir, "Liquidation", "models")
    liq_dir = os.path.join(base_dir, "Liquidation")
    if liq_dir not in sys.path:
        sys.path.insert(0, liq_dir)
```

```
try:
    from features import compute_selling_features
except ImportError:
    from features import compute_features
```

```

validate importable

self.compute_rolling_features = compute_rolling_features
with open(os.path.join(models_dir, "lgb_model.pkl"), "rb") as f:
    self.models["lgb"] = pickle.load(f)
with open(os.path.join(models_dir, "xgb_model.pkl"), "rb") as f:
    self.models["xgb"] = pickle.load(f)
with open(os.path.join(models_dir, "cb_model.pkl"), "rb") as f:
    self.models["cb"] = pickle.load(f)
print("LiquiDation) Successfully loaded all ML LiquiDation models.")
except Exception as e:
    print("LiquiDation) Error loading ML LiquiDation models. (e)")

```

```
def on_tick_update(self, symbol: str, snap: AssetSnapshot,
trade_tracker: Any = None) -> AssetSnapshot:
    with self._lock:
        if snap.price <= 0.0: return snap

        now = time.time()
        open_time = int(now // 900) * 900

        if symbol not in self.candles_history:
            self.candles_history[symbol] = collections.deque(maxlen=1200)

        history = self.candles_history[symbol]
```

```

    if symbol not in self.current_candle or
    self.current_candle[symbol].get("open_time") != open_time:
        prev = self.current_candle.get(symbol)
        if prev and int(prev.get("open_time", 0)) < open_time:
            prev_ot = int(prev["open_time"])
            if not history or int(history[-1].get("open_time", 0)) != prev_ot:
                history.append(dict(prev))

```

```
self.current_candle[symbol] = {
    "open_time": open_time, "open": snap.price, "high":
snap.price,
```

```

snap.price, "low": snap.price, "close": snap.price, "volume": snap.volume,
snap.fut_cvd, "snap fut_cvd", "liq_long": snap.liq_long, "liq_short":
snap.liq_short,
"coins_bid": snap.coins_bid, "coins_ask": snap.coins_ask,
"dollars_bid": snap.dollars_bid, "dollars_ask":
snap.dollars_ask,
"tk_buy_cnt": snap.tk_buy_cnt, "tk_sell_cnt": snap.tk_sell_cnt,
"fp_poc": snap.fp_poc
)
↓

```

```
# Inference only on candle close
if len(history) > 250:
    self.run_inference(symbol, snap_price, trade_tracker)
else:
    candle = self.current_candle[symbol]
    candle["close"] = snap_price
    if snap_price < candle["high"]:
        candle["high"] = snap_price
    if snap_price > candle["low"] or candle["low"] == 0.0:
        candle["low"] = snap_price

    candle["volume"] = snap_volume
    candle["fut_cvd"] = snap_fut_cvd
    candle["liq_long"] = snap_liq_long
    candle["liq_short"] = snap_liq_short
    candle["coins_bid"] = snap_coins_bid
    candle["coins_ask"] = snap_coins_ask
    candle["dollars_bid"] = snap_dollars_bid
    candle["dollars_ask"] = snap_dollars_ask
    candle["tk_buy_cnt"] = snap_tk_buy_cnt
    candle["tk_sell_cnt"] = snap_tk_sell_cnt
    candle["fp_poc"] = snap_fp_poc
```

return snap

```
def _run_inference(self, symbol, current_price, trade_tracker):
    try:
        import pandas as pd
        import polars as pl
        import numpy as np
```

```

compute_rolling_features = self._compute_rolling_features
if compute_rolling_features is None:
    return # models failed to load at init, abort silently

history = list(self.candles_history[symbol])
df = pd.DataFrame(history)

# Build mapped DataFrame matching training column names
mapped_df = pd.DataFrame()
mapped_df['datetime'] = pd.to_datetime(df['open_time'], unit='s')
mapped_df['Open'] = df['open'].astype(float)
mapped_df['High'] = df['high'].astype(float)
mapped_df['Low'] = df['low'].astype(float)
mapped_df['Close'] = df['close'].astype(float)
mapped_df['Volume'] = df['volume'].astype(float)
mapped_df['CVD'] = df['liq_cvd'].astype(float)
mapped_df['Agg_Liq_Long'] = df['liq_long'].astype(float)
mapped_df['Agg_Liq_Short'] = df['liq_short'].astype(float)
mapped_df['Bid_Trades'] = df['tk_buy_cnt'].astype(float)
mapped_df['Ask_Trades'] = df['tk_sell_cnt'].astype(float)
mapped_df['Bid_USD'] = df['dollars_bid'].astype(float)
mapped_df['Ask_USD'] = df['dollars_ask'].astype(float)
mapped_df['Candle_Delta'] = df['coins_bid'].astype(float) -
df['coins_ask'].astype(float)
mapped_df['Delta_USD'] = df['dollars_bid'].astype(float) -
df['dollars_ask'].astype(float)

fp_poc = df.get('fp_poc', df['close']).astype(float)
mapped_df['POC Price'] = np.where((fp_poc == 0.0) |
pd.isna(fp_poc), df['close']).astype(float), fp_poc)

pldf = pl.from_pandas(mapped_df)
pldf = compute_rolling_features(pldf)

# --- 4H Trend Filter: EMA crossover on resampled 4H bars ---
df_4h = pldf.sort('datetime').group_by_dynamic(
    "datetime",
    every="4h",
    closed="left"
) agg([
    pl.col("Close").last().alias("Close")
]) with_columns([
    pl.col("Close").ewm_mean(span=9).alias("ema_9"),
    pl.col("Close").ewm_mean(span=21).alias("ema_21")
]) with_columns([
    pl.when(pl.col("ema_9") > pl.col("ema_21")).then(pl.lit(1))
    .when(pl.col("ema_9") < pl.col("ema_21")).then(pl.lit(-1))
    .otherwise(pl.lit(0)).alias("trend_4h")
])
pldf = pldf.sort("datetime").join_asof(
    df_4h.select(["datetime", "trend_4h"]),
    on="datetime",
    strategy="backward"
)
pldf = pldf.with_columns(pl.col("trend_4h").fill_null(0))

last_row = pldf[-1]
trend_4h = last_row["trend_4h"].item()

# Update latest ATR for trailing stops
atr_val = last_row["atr"].item()
if not np.isnan(atr_val) and atr_val > 0:
    self.latest_atr[symbol] = atr_val

FEATURE_COLS = [
    "trigger_type", "poc_pos", "delta_usd_ratio", "size_ratio",
    "trade_ratio", "liq_long_z_50", "liq_short_z_50",
    "liq_long_z_200", "liq_short_z_200", "cvd_z_10",
    "cvd_z_50", "cvd_z_200", "atr_ratio", "close_to_ema_200"
]

# --- FIX 3: Use z_200 >= 3.0 --- matches generate_labels() training
threshold ---
liq_long_z_200 = last_row["liq_long_z_200"].item()
liq_short_z_200 = last_row["liq_short_z_200"].item()
trigger_type = 0
if liq_long_z_200 >= 3.0 and liq_short_z_200 >= 3.0:
    # Dual-event: take the larger spike, matching generate_labels()
tie-break
    trigger_type = 1 if liq_long_z_200 >= liq_short_z_200 else -1
elif liq_long_z_200 >= 3.0:
    trigger_type = 1
elif liq_short_z_200 >= 3.0:
    trigger_type = -1

if trigger_type == 0:
    return

# --- 4H trend gate: block trades opposing strong trend ---
if (trigger_type == 1 and trend_4h == -1) or (trigger_type == -1 and
trend_4h == 1):
    return

X =
last_row.with_columns(pl.lit(trigger_type).cast(pl.Int8).alias("trigger_type")).select(FEATURE_COLS).to_pandas()
if 'lgb' not in self.models:
    return

lgb_probs = self.models['lgb'].predict_proba(X)
xgb_probs = self.models['xgb'].predict_proba(X)
cb_probs = self.models['cb'].predict_proba(X)

ens_probs = (lgb_probs + xgb_probs + cb_probs) / 3.0
# Class mapping: 0=Breakout(-1), 1=Hold(0), 2=Reversal(+1)
prob_breakout = ens_probs[0, 0]
prob_reversal = ens_probs[0, 2]

# --- FIX 1: Dirichlet logic matching OOS run_simulation() validated
logic ---
# Reversal trade: fade the liquidation spike
# Breakout trade: follow the liquidation direction
direction = 0
confidence = 0.0
if prob_reversal > prob_breakout and prob_reversal >= 0.40:
    # Long-liq event → price bounces UP → go SHORT to fade then
buy dip
    # Short-liq event → price bounces DOWN → go LONG to fade
then sell rip
    direction = -1 if trigger_type == 1 else 1
    confidence = prob_reversal
elif prob_breakout > prob_reversal and prob_breakout >= 0.40:
    # Long-liq event → price continues DOWN → go LONG
(breakout entry)
    # Short-liq event → price continues UP → go SHORT (breakout
entry)
    direction = 1 if trigger_type == 1 else -1
    confidence = prob_breakout

if direction == 0:
    return

atr_val = self.latest_atr.get(symbol, 0.0)
if atr_val <= 0:
    return

# --- FIX 2: Equity curve throttle matching OOS Proposal 3 ---
# OOS: skip entirely at >2.5% equity deviation, half-size at >1.5%
EQUITY_MA_WINDOW = 5
equity_ma = sum(self.recent_capitals[-EQUITY_MA_WINDOW:]) /

```

```

min(len(self.recent_capitals), EQUITY_MA_WINDOW)
current_capital = trade_tracker.current_capital if trade_tracker else
equity_ma
equity_deviation = (equity_ma - current_capital) / equity_ma *
100.0 if equity_ma > 0 else 0.0

if equity_deviation > 2.5:
    return # Hard skip — protect against funded account daily DD
breach

risk_mult = 0.5 if equity_deviation > 1.5 else 1.0

sl_mult = 1.25
tp_mult = 3.0
sl = current_price - sl_mult * atr_val if direction == 1 else
current_price + sl_mult * atr_val
tp = current_price + tp_mult * atr_val if direction == 1 else
current_price - tp_mult * atr_val

if trade_tracker:
    trade_tracker.trigger_entry(
        symbol, "ML_Liquidation_Runner", direction, current_price, sl,
tp, atr_val, macro=0,
vol_regime=0, risk_mult=risk_mult, trail_act=0.5,
regime_val=0
    )

except Exception as e:
    import traceback
    print(f"[Liquidation] {symbol} prediction error:
{e}")

def record_closed_capital(self, capital: float) -> None:
    """Called externally when a Liquidation trade closes to update equity
curve tracker."""
    self.recent_capitals.append(capital)
    if len(self.recent_capitals) > 50:
        self.recent_capitals[-50:]
    ...

### Component B: The Multi-Strategy Trade Tracker
"""python
class Engine1TradeTracker:
    def __init__(self, initial_capital=5000.0):
        self.initial_capital = initial_capital
        self.current_capital = initial_capital
        self.active_trades = {}
        self.last_entry_bar = {}
        self.history = []
        self.on_close_callbacks = [] # callable(strategy, capital) notified on
trade close
        # Always anchored to Engine_1 folder — immune to cwd differences
        self.log_file = os.path.join(
            os.path.dirname(os.path.abspath(__file__)),
            "Engine_1_trade_logs.json"
        )
        self.lock = threading.RLock()
        self.load_history()

    def load_history(self):
        """Load only from this engine's own log — never from Engine 3 whose
pnl_usd uses an incompatible unit-based sizing model"""
        with self.lock:
            if not os.path.exists(self.log_file):
                return
            try:
                with open(self.log_file, 'r', encoding='utf-8') as f:
                    data = json.load(f)
                # Support envelope format with __meta__
                meta = {}
                if isinstance(data, dict) and '__meta__' in data:
                    meta = data['__meta__']
                    data = data['__meta__']
                data = data.get('trades', [])
                if not isinstance(data, list):
                    return
                self.last_entry_bar = meta.get('last_entry_bar', {})
                self.history = [t for t in data if t.get('exit_price')]
                self.current_capital = self.initial_capital + sum(
                    t.get('pnl_usd', 0.0) for t in self.history
                )
                for t in data:
                    if not t.get('exit_price') and t.get('trade_id'):
                        self.active_trades[t['trade_id']] = t
            except Exception as e:
                print(f"[TradeTracker] [ERROR] Failed to load trade history: {e}")

    def save_history(self):
        with self.lock:
            try:
                all_trades = list(self.history) + list(self.active_trades.values())
                envelope = {
                    '__meta__': {
                        'last_entry_bar': dict(self.last_entry_bar)
                    },
                    'trades': all_trades
                }
                tmp = self.log_file + ".tmp"
                with open(tmp, 'w', encoding='utf-8') as f:
                    json.dump(envelope, f, indent=4)
                os.replace(tmp, self.log_file)
            except Exception:
                pass

    def trigger_entry(self, symbol: str, strategy: str, direction: int, entry_price:
float, sl: float, tp: float, atr: float, macro: int, vol_regime: float, risk_mult:
float = 1.0, trail_act: float = 0.5, regime_val: int = 0) -> None:
        with self.lock:
            strategy_trades = {}
            for t in self.active_trades.values():
                if t['strategy']
== strategy:
                    if any(t['symbol'] == symbol for t in strategy_trades):
                        return

            max_concurrent = 3 if regime_val == 1 else 5
            if len(strategy_trades) >= max_concurrent:
                return

            if direction not in (1, -1):
                return

            if direction == 1 and not (sl < entry_price < tp):
                return
            if direction == -1 and not (tp < entry_price < sl):
                return

            stop_dist = abs(entry_price - sl)
            tick_size = globals().get("TICK_SIZES", {}).get(symbol, 0.0001)

            # min_stop with TP factor scaling — matches agent6_exact lines
            min_stop = 5.0 * tick_size
            tp_dist = abs(tp - entry_price)
            if stop_dist < min_stop:
                factor = min_stop / stop_dist
                stop_dist = min_stop
                tp_dist = tp_dist * factor
            if direction == 1:
                sl = entry_price - stop_dist
                tp = entry_price + tp_dist
            else:
                sl = entry_price + stop_dist
                tp = entry_price - tp_dist

```

```

# Guard: reject if stop > 5x ATR --- matches agent6_exact line 292
if atr > 0 and stop_dist > 5.0 * atr:
    return

# Guard: reject if stop_pct < 0.3% --- live tick noise kills tighter
stops
stop_pct = stop_dist / entry_price
if stop_pct < 0.0030:
    return

risk_capital = max(0.0, self.current_capital) * 0.005 * risk_mult

if risk_capital <= 0.0 or stop_dist <= 0:
    return

units = risk_capital / stop_dist

trade_id = f'({symbol})_{"LONG" if direction == 1 else "SHORT"}_{int(time.time())}'
self.active_trades[trade_id] = {
    "trade_id": trade_id,
    "symbol": symbol,
    "strategy": strategy,
    "direction": direction,
    "entry_price": entry_price,
    "entry_time": time.strftime("%Y-%m-%d %H:%M:%S"),
    "entry_timestamp": time.time(),
    "sl": sl,
    "tp": tp,
    "units": units,
    "live_pnl_pct": 0.0,
    "live_pnl_usd": 0.0,
    "atr": atr,
    "macro": macro,
    "vol_regime": vol_regime,
    "sl_dist": stop_dist,
    "trail_act": trail_act,
    "trail_buf": 0.5
}
self.save_history()

def update_live_pnl(self, symbol: str, current_price: float) -> None:
    with self.lock:
        trades_for_symbol = [t for t in self.active_trades.values() if
            t['symbol'] == symbol]
        for trade in trades_for_symbol:
            direction = trade['direction']
            entry_price = trade['entry_price']

            pnl_pct = (current_price - entry_price) / entry_price * 100.0 if
            direction == 1 else (entry_price - current_price) / entry_price * 100.0
            pnl_usd = trade['units'] * (current_price - entry_price) * direction

            trade['live_pnl_pct'] = pnl_pct
            trade['live_pnl_usd'] = pnl_usd

def check_exits(self, symbol: str, current_price: float,
    current_atr_or_dict: Any = 0.0) -> None:
    with self.lock:
        trades_for_symbol = [t for t in self.active_trades.values() if
            t['symbol'] == symbol]
        any_closed = False
        for trade in trades_for_symbol:
            direction = trade['direction']
            sl = trade['sl']
            tp = trade['tp']
            entry_price = trade['entry_price']

            # Resolve cl ↓ ATR based on strategy if a dict is passed
            if isinstance(current_atr_or_dict, dict):
                strategy = trade.get('strategy', 'AlphaSqueezer_V17')
                current_atr = float(current_atr_or_dict.get(strategy, 0.0) or 0.0)
            else:
                current_atr = float(current_atr_or_dict or 0.0)

            # --- Trailing Stop Logic (Parity with agent6_exact.py) ---
            sl_dist = trade.get('sl_dist')
            trail_act = trade.get('trail_act')
            trail_buf = trade.get('trail_buf', 0.5)

            if sl_dist and trail_act and current_atr > 0:
                if direction == 1:
                    cur_r = (current_price - entry_price) / sl_dist
                    if cur_r >= trail_act:
                        ns = current_price - trail_buf * current_atr
                        if ns > sl:
                            trade['sl'] = ns
                            sl = ns
                else:
                    cur_r = (entry_price - current_price) / sl_dist
                    if cur_r >= trail_act:
                        ns = current_price + trail_buf * current_atr
                        if ns < sl:
                            trade['sl'] = ns
                            sl = ns

            should_close = False
            reason = ""

            # --- MAX_BARS Timeout Exit (Parity with agent6_exact.py) ---
            # 96 bars of 15m = 24 hours (86400 seconds)
            elapsed_time = time.time() - trade.get('entry_timestamp',
            time.time())
            if elapsed_time >= 86400:
                should_close = True
                reason = "TIMEOUT"

            if not should_close:
                if direction == 1:
                    if current_price <= sl:
                        should_close = True
                        reason = "SL"
                    elif current_price >= tp:
                        should_close = True
                        reason = "TP"
                else:
                    if current_price >= sl:
                        should_close = True
                        reason = "SL"
                    elif current_price <= tp:
                        should_close = True
                        reason = "TP"

            if should_close:
                trade['exit_price'] = current_price
                trade['exit_time'] = time.strftime("%Y-%m-%d %H:%M:%S")
                trade['exit_reason'] = reason

                entry_price = trade['entry_price']
                pnl_pct = (current_price - entry_price) / entry_price * 100.0 if
                direction == 1 else (entry_price - current_price) / entry_price * 100.0
                pnl_pct -= 0.06 # Modeled commissions & slippage cost

                # Compute Net PnL USD based on fixed units and entry
                transaction cost
                pnl_usd = (trade['units'] * (current_price - entry_price) *
                direction) - (trade['units'] * entry_price * 0.0006)
                ↓
                trade['pnl_pct'] = pnl_pct
                trade['pnl_usd'] = pnl_usd

```



```

        data["pnl_usd"] = pnl_usd

        self.history.append(trade)
        self.current_capital += pnl_usd
        del self.active_trades[trade["trade_id"]]
        any_closed = True
        # Notify registered predictors so they can update their equity

trackers

        closed_strategy = trade.get("strategy", "")
        for cb in self.on_close_callbacks:
            try:
                cb(closed_strategy, self.current_capital)
            except Exception:
                pass

        if any_closed:
            self.save_history()
        ...

### Component C: The Tick Snapshot Store
"""python
class SnapshotStore:
    def __init__(self, symbols: List[str], predictor: LiveStrategyPredictor =
None, liquidation_predictor: Any = None, trade_tracker: Any = None):
        self._data: Dict[str, AssetSnapshot] = {}
        self.symbols = symbols
        self._locks = {}
        self._seq = 0
        self.predictor = predictor
        self.liquidation_predictor = liquidation_predictor
        self.trade_tracker = trade_tracker

    async def update(self, symbol: str, **patch: Any) -> None:
        if symbol not in self._data:
            return
        async with self._locks[symbol]:
            cur = self._data[symbol]
            clean_patch = {}
            for k, v in patch.items():
                if not hasattr(cur, k):
                    continue
                if k in ("price", "open", "high", "low", "close"):
                    tv = finite_float_or_none(v)
                    if tv is None:
                        continue
                    if k == "price" and tv <= 0.0:
                        continue
                    clean_patch[k] = tv
            else:
                clean_patch[k] = v

        if not clean_patch:
            return

        self._seq += 1
        new_snap = dataclasses.replace(cur, seq=self._seq,
ts_ns=time.time_ns(), **clean_patch)

        price_updated = "price" in clean_patch

        if self.trade_tracker and price_updated:
            atr_dict = {}
            if self.predictor:
                cached = self.predictor._cached_signal.get(symbol, {})
                atr_dict["AlphaSqueezer_V17"] = cached.get("atr_val", 0.0)
            if getattr(self, "liquidation_predictor", None):
                atr_dict["ML_Liquidation_Runner"] =
getattr(self.liquidation_predictor, "latest_atr", {}).get(symbol, 0.0)
            self.trade_tracker.check_exits(symbol, new_snap.price, atr_dict)
            self.trade_tracker.update_live_pnl(symbol, new_snap.price)

        if self.predictor and new_snap.price > 0.0:
            new_snap = await
asyncio.to_thread(self.predictor.on_tick_update, symbol, new_snap,
self.trade_tracker)

        if getattr(self, "liquidation_predictor", None) and new_snap.price >
0.0:
            new_snap = await
asyncio.to_thread(self.liquidation_predictor.on_tick_update, symbol,
new_snap, self.trade_tracker)
        self._data[symbol] = new_snap
    ...

...

### 3. AUDIT OBJECTIVES & SEVERE PITFALL CHECKLIST
Please analyze the code above and respond to the following critical audit
vectors.

### Vector 1: Concurrency and Thread Safety ('asyncio' vs. 'threading')
- **Context:** The main execution loop is asynchronous ('asyncio'), but the
UI loop, 'trade_tracker', and the prediction models run in synchronous
blocks or threads ('asyncio.to_thread').
- **Audit Questions:**
    1. Does the use of 'threading.RLock()' in 'Engine1TradeTracker'
('self.lock') prevent race conditions when 'check_exits' and 'trigger_entry'
are called concurrently by both strategy threads?
    2. In 'SnapshotStore.update()', 'self._locks[symbol]' (an 'asyncio.Lock()')
prevents concurrent tick updates for the same symbol. However, when we
offload model inference to
'asyncio.to_thread(self.predictor.on_tick_update...)', is it possible that
'new_snap' updates on disk or state are interleaved unsafely?
    3. Analyze the thread safety of 'self.recent_capitals' and 'self.latest_atr'
writes inside 'LiveLiquidationPredictor' since they lack explicit lock
protection during list appends and dict updates.

### Vector 2: Trailing Stop Parity & ATR Routing Mismatch
- **Context:** 'SnapshotStore' passes a strategy-to-ATR dictionary
'atr_dict' to 'check_exits(symbol, new_snap.price, atr_dict)'.
- **Audit Questions:**
    1. Verify the dictionary routing logic inside 'check_exits': If a trade
strategy is 'ML_Liquidation_Runner', does it successfully fetch and apply
the ATR computed via 'compute_rolling_features'?
    2. What happens if the 'latest_atr' dictionary has no entry for 'SUIUSD'
because the history for SUI hasn't reached 250 bars yet? Will it default to
0.0, and if so, does 'check_exits' handle this safely without crashing or
leaving stops wide open?
    3. Ensure there is no risk of one strategy's ATR value being used for the
other strategy's trailing stop calculations.

### Vector 3: Risk Controls & Drawdown Protection (Funded Account
Rules)
- **Context:** The funded account has a 5% daily limit.
- **Audit Questions:**
    1. Evaluate 'LiveLiquidationPredictor's equity MA circuit breaker. In a
rapid series of losing liquidation trades, is there a latency window or
lookback gap where 'trade_tracker.current_capital' is updated, but the next
trade is triggered "before" 'record_closed_capital' updates
'self.recent_capitals'?
    2. Does the hard skip ('if equity_deviation > 2.5: return') execute fast
enough to block pending entry orders in the execution loop?
    3. 'risk_capital' is calculated as 'max(0.0, self.current_capital) * 0.005 *
risk_mult'. If 'current_capital' drops, the absolute trade size drops. Is this
dynamic scaling mathematically bulletproof against a series of 5
consecutive SL hits?

### Vector 4: Data Parity & Lookahead Bias
- **Context:** The model is trained on data processed by 'polars' from
historical Parquet files.
- **Audit Questions:**

```

```

1. Contrast the live data mapping in
'LiveLiquidationPredictor_run_inference' (which uses a pandas
DataFrame built from a deque of 'collections deque' and translates it to
polars via 'pl.from_pandas') with 'features.py' training code. Is there any
potential lookahead bias or data translation discrepancy (e.g. naive time
zone handling, floating-point rounding, or scaling differences)?
2. Verify that the 4H Trend Filter resamples and applies the EMA
crossover matching the backtest perfectly without lookahead leakage.

### Vector 5: Strategy Isolation
- **Context:** 'active_trades' is keyed by 'trade_id'.
- **Audit Questions:**
  1. Confirm that 'symbol in strategy_trades' check in 'trigger_entry' allows
  'AlphaSqueezer_V17' and 'ML_Liquidation_Runner' to hold positions in
  opposite directions on the same symbol (e.g. SUI LONG and SUI SHORT
  concurrently) without modifying or exiting the wrong position.
  2. Verify that 'check_exits' correctly targets and deletes only the exact
  'trade[trade_id]' from 'self.active_trades' on exit triggers.

```

Executive verdict

The integration is **partially thread-safe**, but I would **not run this as-is on a 5% daily-DD funded MT5 account**. The biggest blockers are not the basic RLock usage; they are:

1. **No hard global daily/total DD governor** using broker equity + unrealized PnL.
2. **Predictors can run on stale price** because `SnapshotStore.update()` calls them on non-price updates.
3. **Liquidation strategy uses live candles built from sampled close ticks, not true OHLC**, causing training/live feature drift.
4. **ML_Liquidation_Runner trend gate is applied to trigger_type, not final trade direction**, which can block/allow the wrong side.
5. **Cross-strategy trade ID collision is possible** because `trade_id` omits strategy and uses `int(time.time())`.
6. **Liquidation predictor state is not fully lock-protected**, especially `recent_capitals`, `latest_atr` reads, and `trade_tracker.current_capital` reads.
7. **Security issues**: hardcoded Coinglass credentials, `pickle.load()` model loading, persistent Chrome session + remote debugging port, swallowed persistence errors.

Vector 1 — Concurrency and thread safety

1. Does `Engine1TradeTracker.self.lock` prevent concurrent `check_exits()` / `trigger_entry()` races?

Mostly yes for `active_trades`, `history`, and `current_capital`.

`trigger_entry()`, `check_exits()`, `update_live_pnl()`, `get_stats()`, `load_history()`, and `save_history()` use the same threading.RLock(). This prevents direct concurrent mutation corruption when Alpha and Liquidation threads both call into the tracker.

However, important gaps remain:

Good

- `active_trades` mutation is serialized.
- `current_capital` updates on close are serial.
- `save_history()` is called under the same lock.
- RLock avoids deadlock when `trigger_entry()` calls `save_history()` while already holding the lock.

Problems

A. `trade_tracker.current_capital` is read outside the lock

In `LiveLiquidationPredictor._run_inference()`:

```
current_capital = trade_tracker.current_capital if trade_tracker else equity_ma
```

That read is not protected by `trade_tracker.lock`. In CPython this usually will not corrupt memory, but it is not semantically atomic relative to concurrent closes/entries.

Fix: move all pre-trade risk checks into `Engine1TradeTracker.trigger_entry()` under `self.lock`, or expose a locked getter.

B. `last_entry_bar` is modified outside `Engine1TradeTracker.lock`

In the full Alpha code, this occurs outside the tracker lock:

```
trade_tracker.last_entry_bar[symbol] = last_bar_time
```

Also, `trigger_entry()` saves history before this update, so `last_entry_bar` may not persist immediately.

Fix: pass `entry_bar` into `trigger_entry()` and update `last_entry_bar` under the lock before `save_history()`.

C. `trade_id` can collide across strategies

Current ID:

```
trade_id = f"{symbol}_{'LONG' if direction == 1 else 'SHORT'}_{int(time.time())}"
```

If Alpha and Liquidation both enter `SUIUSD1 LONG` in the same second, the second write overwrites the first in `active_trades`.

Fix:

```
import uuid

trade_id = (
    f"{strategy}_{symbol}_{'LONG' if direction == 1 else 'SHORT'}_"
    f"{time.time_ns()}_{uuid.uuid4().hex[:8]}"
)
```

This is a high-priority fix.

2. Is `SnapshotStore.update()` safe when using `asyncio.to_thread()`?

For same-symbol updates, **yes, mostly**. This lock:

```
async with self._locks[symbol]:
```

prevents two concurrent updates for the same symbol from interleaving. The assignment:

```
self._data[symbol] = new_snap
```

happens after both predictors finish.

However, there are behavioral problems.

Problem A — per-symbol lock is held while model inference runs

This is safe, but it can create latency/backpressure. If inference takes 1–3 seconds, all updates for that symbol wait.

This is not a memory-safety issue, but for a live execution engine it can delay price/exits.

Problem B — predictors run on non-price updates

This is serious.

Current logic:

```
if self.predictor and new_snap.price > 0.0:
    new_snap = await asyncio.to_thread(...)

if self.liquidation_predictor and new_snap.price > 0.0:
    new_snap = await asyncio.to_thread(...)
```

This runs predictors whenever any field updates, including `fp_delta`, `oi`, `funding`, etc., as long as the snapshot already has a nonzero price.

If the price feed stalls but Binance footprint keeps updating, then:

- `ts_ns` becomes fresh,
- `renderer` shows the row as fresh,
- predictors can run using stale price,
- `check_exits()` does **not** run because `price_updated` is false.

That is a funded-account risk.

Fix: maintain `price_ts_ns` separately and only run entry/exit logic when price is fresh.

Minimal rule:

```
price_is_fresh = (
    new_snap.price > 0.0 and
    getattr(new_snap, "price_ts_ns", 0) > 0 and
    time.time_ns() - new_snap.price_ts_ns < 3_000_000_000
)

if price_updated and price_is_fresh:
    check_exits(...)
    run_predictors(...)
```

Better: drive strategy inference only from confirmed 15m candle-close events, not arbitrary field updates.

3. Are `recent_capitals` and `latest_atr` thread-safe?

Not fully.

`latest_atr`

Written here under `LiveLiquidationPredictor._lock`:

```
self.latest_atr[symbol] = atr_val
```

Read here without that lock:

```
getattr(self.liquidation_predictor, 'latest_atr', {}).get(symbol, 0.0)
```

CPython dictionary reads/writes are generally memory-safe due to the GIL, but this is not a robust synchronization contract.

`recent_capitals`

Read in `_run_inference()` while holding `self._lock`, but appended in `record_closed_capital()` without holding that lock:

```
self.recent_capitals.append(capital)
```

Again, usually memory-safe in CPython, but semantically racy.

Recommended fix

Refactor so predictors do not call `trade_tracker.trigger_entry()` while holding their own locks. Otherwise, adding locks can create future lock-order deadlocks.

Preferred pattern:

```
def on_tick_update(...):
    trade_intent = None

    with self._lock:
        # update candles, compute inference
        trade_intent = self._maybe_build_trade_intent(...)

    # outside predictor lock
    if trade_intent and trade_tracker:
        trade_tracker.trigger_entry(*trade_intent)

    return snap
```

Then you can safely lock these:

```
def get_latest_atr(self, symbol: str) -> float:
    with self._lock:
        return float(self.latest_atr.get(symbol, 0.0) or 0.0)

def record_closed_capital(self, capital: float) -> None:
    with self._lock:
        self.recent_capitals.append(float(capital))
        self.recent_capitals = self.recent_capitals[-50:]
```

Vector 2 — Trailing stop parity and ATR routing

1. Does `ML_Liquidation_Runner` fetch its own ATR?

Yes, assuming the strategy string is exactly:

```
"ML_Liquidation_Runner"
```

In `SnapshotStore.update()`:

```
atr_dict["M_Liquidation_Runner"] = liquidation_predictor.latest_atr.get(symbol, 0.0)
```

In `check_exits()`:

```
strategy = trade.get('strategy', 'AlphaSqueezer_V17')
current_atr = float(current_atr_dict.get(strategy, 0.0) or 0.0)
```

So if the trade has:

```
trade["strategy"] == "M_Liquidation_Runner"
```

it receives the liquidation ATR.

2. What if `latest_atr` has no entry for `SUIUSD`?

Then:

```
current_atr = 0.0
```

This is safe in the sense that it does not crash.

Also, stops are not "wide open." SL/TP checks still work because they are independent of ATR:

```
if current_price <= sl:
    close
elif current_price >= tp:
    close
```

But trailing is disabled because this condition fails:

```
if sl_dist and trail_act and current_atr > 0:
```

So the trade remains protected by its original SL/TP, but trailing parity is lost.

Recommended fallback:

```
current_atr = float(current_atr_dict.get(strategy, 0.0) or trade.get("atr", 0.0) or
```

That protects active trades after restart, before `latest_atr` has been recomputed.

3. Can one strategy's ATR leak into the other?

With exact strategy strings, no.

However, risks remain:

- Old log entries without strategy default to Alpha.
- A typo in strategy name silently gives ATR 0.0.
- `latest_atr` read is unlocked.
- After restart, `latest_atr` is empty until the liquidation predictor warms up again.

Use constants:

```
STRAT_ALPHA = "AlphaSqueezer_V17"
STRAT_LIQ = "M_Liquidation_Runner"
```

and avoid raw string duplication.

Vector 3 — Risk controls and funded DD protection

1. Is there a latency window before `record_closed_capital()` updates `recent_capitals`?

For the same symbol update path, the order is good:

```
check_exits()
update_live_pnl()
predictor.on_tick_update()
liquidation_predictor.on_tick_update()
```

So if a trade closes on that tick, `current_capital` updates before new inference runs.

But across symbols/threads, there are still gaps:

- `current_capital` is read without `trade_tracker.lock`.
- `recent_capitals` is updated without `LiveLiquidationPredictor._lock`.
- Multiple symbol inference threads can pass the equity gate using the same stale capital snapshot.
- The equity circuit breaker only affects Liquidation, not Alpha.
- It does not include unrealized PnL.

So the current circuit breaker is not a hard funded-account protection mechanism.

2. Does if `equity_deviation > 2.5`: return `block pending orders fast enough`?

It blocks only the local liquidation entry decision before `trigger_entry()`.

It does **not**:

- cancel existing pending MT5 orders,
- close open positions,
- block Alpha entries,
- account for unrealized losses,
- enforce broker-side equity drawdown,
- prevent another concurrent thread from already having passed the check.

For a funded account, the hard DD gate should live inside a central order/risk manager, not inside one predictor.

3. Is dynamic 0.5% sizing bulletproof against 5 SL hits?

For sequential, perfectly filled SLs, roughly yes.

Ignoring fees:

```
capital_after_5 = capital * (0.995)**5 ≈ 97.52%
DD ≈ 2.48%
```

But your PnL USD calculation subtracts cost:

```
pnl_usd = gross_pnl - (units * entry_price * 0.0006)
```

At the minimum stop filter:

```
stop_pct >= 0.0030
```

Copy

the fee/slippage cost is:

```
0.0006 / 0.0030 = 20% of stop risk
```

Copy

So a 0.5% risk trade can lose about:

```
0.5% * 1.20 = 0.6%
```

Copy

Five losses ~ 3%.

That is still below 5%, but only if trades are sequential.

The real danger is concurrency:

- Alpha can hold up to 5 trades.
- Liquidation can hold up to 5 trades.
- Total possible simultaneous trades = 10.
- $10 \times 0.5\%$ gross risk = 5%.
- With modeled cost, this can become ~ 6%.
- With slippage/gaps, worse.

Therefore, the system is **not mathematically bulletproof** against funded DD breach.

Required funded-account risk layer

Add a global guard inside `Engine1TradeTracker.trigger_entry()` under `self.lock`:

- daily starting equity,
- total starting equity,
- current broker equity,
- unrealized PnL,
- realized PnL,
- open stop risk,
- projected new trade risk,
- max per-symbol risk,
- max correlated crypto basket risk,
- emergency halt flag.

Example logic:

```
equity = self.current_capital + sum(t.get("live_pnl_usd", 0.0) for t in
self.active_trades.values())
open_stop_risk = sum(t["units"] * abs(t["entry_price"] - t["sl"])) for t in
self.active_trades.values())

projected_risk = open_stop_risk + risk_capital

if daily_dd_pct >= 4.0:
    return

if total_dd_pct >= 8.0:
    return

if projected_risk > equity * 0.025:
    return
```

For a 5% hard daily breach, I would not allow projected worst-case open risk above 2–3%.

Vector 4 — Data parity and lookahead bias

1. Live mapping vs training data

The column mapping broadly resembles training format, but there are several serious parity risks.

Major issue A — live OHLC is not true OHLC

In live mode, `LiveLiquidationPredictor` builds candles from sampled `snap.price`:

```
"open": snap.price,
"high": snap.price,
"low": snap.price,
"close": snap.price,
```

Then high/low are updated from sampled prices.

This is not equivalent to historical OHLC from `Parquet/TradingView/Binance` candles. `ATR`, `poc_pos`, `atr_ratio`, `close_to_ema_200`, and other rolling features can materially diverge.

Fix: feed true 15m OHLCV into the predictor, either from `Binance` klines or from `Coinglass` chart OHLC extraction. Do not reconstruct OHLC from 0.5s sampled closes.

Major issue B — Liquidation history is not seeded

Alpha loads disk history:

```
predictor.load_history_from_disk()
```

Liquidation does not.

So `ML_Liquidation_Runner` starts cold and needs more than 250 live bars before inference. That is over 62 hours on 15m candles.

Also, 250 bars is not really enough for stable 4H EMA21:

```
21 * 4H = 84H = 336 15m bars
```

So the liquidation warmup should be at least 400–850 bars, and preferably loaded from the same seed workbook.

Major issue C — predictors can run on stale price

As noted above, non-price updates can trigger inference using old price. This can create fake candle transitions and bad entry prices.

Add per-field timestamps, especially `price_ts_ns`.

Major issue D — local wall-clock candle boundaries

Live candles are keyed by:

```
open_time = int(time.time() // 900) * 900
```

[Copy](#)

This depends on local machine time. If local clock drifts or the feed lags, your candle boundaries can differ from exchange/backtest boundaries.

Prefer exchange candle open time from kline data.

Major issue E — possible model class-order mismatch

You assume:

```
ens_probs[0, 0] = breakout  
ens_probs[0, 2] = reversal
```

[Copy](#)

But `predict_proba()` column order depends on model class ordering.

You should assert this explicitly.



```
print(model.classes_)
```

[Copy](#)

or save a metadata file mapping class index to semantic label.

Otherwise, the liquidation strategy can silently invert breakout/reversal.

Major issue F — liquidation direction mapping needs verification

Current code:

```
if prob_reversal > prob_breakout:  
    direction = -1 if trigger_type == 1 else 1  
elif prob_breakout > prob_reversal:  
    direction = 1 if trigger_type == 1 else -1
```

[Copy](#)

Given the comments:

```
Long-liq event → price bounces UP
```

[Copy](#)

a reversal after long liquidations would normally imply LONG, not SHORT.

So either:

- your `trigger_type` semantics are opposite of the names,
- your labels are opposite of the comments,
- or the direction mapping is inverted.



This needs a hard unit test against `generate_labels()` and `run_simulation()`.

Major issue G — 4H trend gate is applied before final direction

Current gate:

```
if (trigger_type == 1 and trend_4h == -1) or (trigger_type == -1 and trend_4h == 1):  
    return
```

[Copy](#)

But final direction can be reversal or breakout. Therefore, gating by `trigger_type` can block/allow the wrong side.

If the intent is "block trades opposing the 4H trend," the gate should happen after direction is chosen:

```
if (direction == 1 and trend_4h == -1) or (direction == -1 and trend_4h == 1):  
    return
```

[Copy](#)

This is a high-priority behavioral audit item.

2. Does the 4H EMA filter leak lookahead?

In live inference, it uses only history, and history contains closed 15m bars. So for the final live row, there is no future data in the dataframe.

However, parity with backtest depends on how the backtest calculated the same filter.

This code:



```
df_4h = pldf.sort("datetime").group_by_dynamic(  
    "datetime",  
    every="4h",  
    closed="left"  
)  
.agg([  
    pl.col("Close").last().alias("Close")  
])
```

[Copy](#)

For the latest live row, this gives the latest known partial 4H close.

But if the backtest computed this on the full dataset and joined the final 4H close back to every 15m row inside the same 4H bucket, then the backtest would leak future intra-4H closes.

To avoid ambiguity, decide one of these:

Option A — use partial current 4H bar

Then backtest must compute the 4H trend point-in-time at each 15m step.

Option B — use only completed 4H bars

Then shift the 4H trend by one completed 4H bar before `join_asof()`.

For funded execution, Option B is safer and easier to audit.

Vector 5 — Strategy isolation

1. Can Alpha and Liquidation hold opposite positions on the same symbol?

In the internal tracker, yes.



This logic blocks only same-strategy same-symbol duplication.

```
strategy_trades = [t for t in self.active_trades.values() if t['strategy'] == strategy]  
  
if any(t['symbol'] == symbol for t in strategy_trades):  
    return
```

[Copy](#)

Therefore:

- Alpha can hold SUIUSD `LONG`.
- Liquidation can hold SUIUSD `SHORT`.
- Both can coexist in `active_trades`.

However, two caveats:

Caveat A — trade ID collision

As mentioned, same symbol, same direction, same second, different strategies can overwrite each other.

Fix trade ID immediately.

Caveat B — MT5 account mode

If the MT5 account is **netting**, opposite positions on the same symbol cannot truly coexist. The broker will net/reduce/reverse the position, while your tracker thinks two independent trades exist.

You need to confirm the account is **hedging mode**. Also use separate MT5 magic numbers/comments per strategy.

2. Does `check_exits()` delete only the correct trade?

Yes.

It builds a list snapshot:

```
trades_for_symbol = [t for t in self.active_trades.values() if t['symbol'] == symbol]
```

Then deletes by exact ID:

```
del self.active_trades[trade['trade_id']]
```

That is correct and safe against dictionary mutation during iteration.

If both long and short exist on the same symbol, the same price tick can close one or both depending on each trade's SL/TP. That behavior is logically correct.

Additional security findings

Critical — hardcoded credentials

The full script contains fallback Coinglass credentials in source code.

That is a severe operational security issue.

Fix:

```
email = os.environ["COINGLASS_EMAIL"]
password = os.environ["COINGLASS_PASSWORD"]
```

Do not provide real default values. Rotate the exposed password.

Also avoid screenshots after filling login forms.

High — unsafe `pickle.load()`

This is dangerous.

```
pickle.load(f)
```

Pickle can execute arbitrary code if the file is modified.

Only load model artifacts from trusted, signed, permission-restricted paths. Ideally add SHA256 verification before loading.

High — `sys.path.insert(0, liq_dir)`

This allows import hijacking via a malicious `Features.py`.

Safer: import by absolute file path using `importlib.util.spec_from_file_location()` or package the module properly.

High — Chrome remote debugging port

This flag:

```
"--remote-debugging-port=9222"
```

can expose browser/session control to local malware or other users on the machine.

Remove it unless strictly needed. If needed, bind to loopback explicitly and restrict OS access.

Medium — swallowed persistence errors

This silently ignores log-save failures:

```
except Exception:
    pass
```

For a live trading system, that is dangerous. If logs fail, restart recovery can be wrong.

At minimum, print and halt new entries on persistence failure.

Minimum hardening plan before funded deployment

P0 — must fix before live funded use

1. Add a **global risk governor** inside `Engine1TradeTracker.trigger_entry()`:
 - daily DD,
 - total DD,
 - unrealized PnL,
 - open stop risk,
 - projected portfolio risk,
 - broker equity,
 - emergency halt.
2. Fix `trade_id` uniqueness.
3. Prevent predictors from running on stale/non-price updates.
4. Add `price_ts_ns` and preferably per-field timestamps.
5. Feed true 15m OHLCV into both predictors.
6. Seed `LiveLiquidationPredictor` from historical workbook.
7. Move the 4H trend gate after final direction selection, unless backtest explicitly proves current behavior.
8. Assert model `predict_proba()` class ordering.
9. Remove hardcoded credentials and rotate exposed secrets.
10. Remove or secure `pickle.load()` and Chrome remote debugging.

P1 — strong recommendations

1. Refactor predictors to return trade intents instead of calling `trade_tracker.trigger_entry()` while holding predictor locks.
2. Add locked-pattern test functions: ATP and empty state.

- 2. Add locked generators for liquidation ATR and equity state.
- 3. Use broker bid/ask/mid data for exits and PnL, not only scraped Coinglass price.
- 4. Add per-strategy MT5 magic numbers.
- 5. Add a point-in-time parity test:
 - feed historical candles row-by-row into live predictor,
 - compare generated features to backtest features,
 - compare model probabilities,
 - compare final trade decisions.

Final assessment

The current code is a solid prototype integration, but it is **not yet funded-account safe**. The base RLock design protects the trade dictionary reasonably well, and ATR routing mostly works, but the system lacks a true hard DD governor, has stale-price inference risk, has cross-strategy trade ID collision risk, and has significant training/live data parity risks.

The highest-impact fixes are: **central risk manager, unique trade IDs, fresh-price gating, true OHLCV candle construction, liquidation history seeding, and explicit model/4H-trend parity tests.**

📄 👍 💬 ...

Your credit balance is negative. This run was stopped. Please top up to continue. [upgrade](#)

📄 👍 💬 ...

AI Chat

Stop paying for AI five times →

Ask anything, create anything



GPT-5.5 Pro

